



Intelligent Multi-Platform Coding Performance Analyzer

SE-801 Final Presentation · Software Project Lab III

Abdus Salam Islam Badhon · BSSE-1401

Supervisor: Mohd. Zulfiquar Hafiz, Professor

Institute of Information Technology, University of Dhaka

github.com/Badhon1401/SPL-3



Agenda

- 1 Problem, motivation and objectives
- 2 Functional and non-functional requirements
- 3 Modelling: use cases, data, architecture
- 4 Component-level design, algorithms, alternatives
- 5 Interface design and the running application
- 6 Implementation and AI engineering
- 7 Testing: strategy, test cases and results
- 8 Conclusion and deliverables

Problem Statement

Why CodePulse

- Competitive programmers practise on several judges, each with its own isolated statistics
- No single place shows combined performance, acceptance rate or activity across platforms
- No automated weakness detection that spans platforms (e.g. repeated Dynamic Programming failures on Codeforces and LeetCode)
- Choosing what to practise next is ad hoc; practice is not guided by measured weaknesses
- Behavioural patterns — declining activity, comfort-zone stagnation, blind-spot topics — are invisible to the learner

Motivation and Objectives

- Mainstream platforms excel at hosting problems, but give little structured feedback on how to improve
- Goal: turn raw submission data into actionable, personalised guidance
- Aggregate Codeforces, LeetCode, AtCoder and CodeChef in one dashboard
- Compute a weighted CodePulse Rating, streaks, topic weakness and hidden insights
- Recommend unsolved problems from weak topics, with a solved / dismiss feedback loop
- AI Coach: free-text goals answered by LLMs grounded in the user's real history
- Secure (BCrypt + JWT), bounded data growth (300 submissions per user), resilient to external failures

Scope and Deliverables

In scope

- Registration, login, profile handles
- Sync of 4 platforms (200 / sync, 6-hourly job)
- Combined rating, streaks, heatmap, insights
- Curated and AI recommendations
- Next.js UI with charts, Spring Boot REST API
- Admin user listing

Deliverables

- Full-stack application and 18 REST endpoints
- PostgreSQL schema (7 tables + join table)
- Final report (48 pages) with user manual
- Public repository: Badhon1401/SPL-3
- 17 automated end-to-end tests, all passing
- Installer: not applicable (run from source)

Positioning of CodePulse

Capability	Single-platform profile	CodePulse
Cross-platform dashboard	No	Yes — four platforms
Unified rating	No	Yes — weighted CodePulse Rating
Topic weakness scoring	Limited	Automatic failure-rate score per topic
Hidden insights	No	6–10 generated findings
Personalised recommendations	Generic lists	Weakness-based + AI Coach with history
Natural-language practice goals	No	Yes — Groq / Mistral / OpenRouter

Functional Requirements (1/3)

Authentication, synchronisation

ID	Requirement
FR-1	Register with username, email, password; BCrypt-hashed storage
FR-2	Login and receive a JWT access token (24-hour expiry)
FR-3	Store and update Codeforces, LeetCode, AtCoder, CodeChef handles
FR-4	Handles always read fresh from the database on profile / dashboard load
FR-5	Fetch up to 200 recent submissions per platform per sync
FR-6	Deduplicate by platformSubmissionId before storage
FR-7	Prune to the most recent 300 submissions per user
FR-8	Automatic sync every 6 hours plus manual trigger

Functional Requirements (2/3)

Analytics and recommendations

ID	Requirement
FR-9	Combined CodePulse Rating from all platforms (weighted average)
FR-10	Current streak, longest streak and active days across platforms
FR-11	Topic-level weakness scores (failure rate per topic)
FR-12	Generate 6–10 hidden insights automatically
FR-13	Per-platform statistics next to combined statistics
FR-14	Select up to 10 unsolved problems from weakest topics in the difficulty range
FR-15	New recommendations replace the previous ones
FR-16	Mark a recommendation solved (feedback loop) or dismiss it

Functional Requirements (3/3)

AI Coach

ID	Requirement
FR-17/18	Free-text prompt; context payload (rating, weaknesses, stats, recent activity) sent to the LLM providers
FR-19	Structured JSON: recommendations, coach insight, focus areas
FR-20/21	Persist sessions; latest shown on load; regeneration replaces the active session
FR-22	Mark AI items solved or dismiss
FR-23	Groq, Mistral AI, OpenRouter — used only if keyed; ordered automatically
FR-24	Run-time model discovery, best-first trial, cooldown for failed models
FR-25	Accept only valid JSON with a non-empty recommendations array
FR-26	Keep only URLs on the four platforms; otherwise a search link
FR-27	Authenticated self-test endpoint <code>/api/ai/health/{provider}</code>

Non-Functional Requirements

ID	Attribute	Requirement	Measure
NFR-1	Performance	Analytics for up to 300 submissions	< 2 s
NFR-2	Security	JWT on all but /api/auth/**; no plaintext passwords	401 without token
NFR-3	Reliability	One platform failing must not crash the sync job	Error logged; others continue
NFR-4	Scalability	Bounded growth; asynchronous sync	≤ 300 rows per user
NFR-5	Usability	Skeletons, toasts, empty states	Manual acceptance
NFR-6	Maintainability	Each platform sync is an independent service	New platform = 1 service + 1 field
NFR-7	AI correctness	JSON-only prompt; AiJsonUtil validation	Tolerates schema additions
NFR-8	Portability	Java 17, PostgreSQL 15, Node.js 18	Any OS
NFR-9	AI availability	Per-model timeouts; ≤ 10 OpenRouter tries / 90 s; no DB transaction during LLM call	Friendly error, no stack trace

Stakeholders, Assumptions and Constraints

Stakeholders

- Competitive programmers — primary users
- CS students — interview and contest prep
- Supervisor / evaluator — design quality
- Developer / maintainer — clean architecture
- Platform providers — fair-use API access

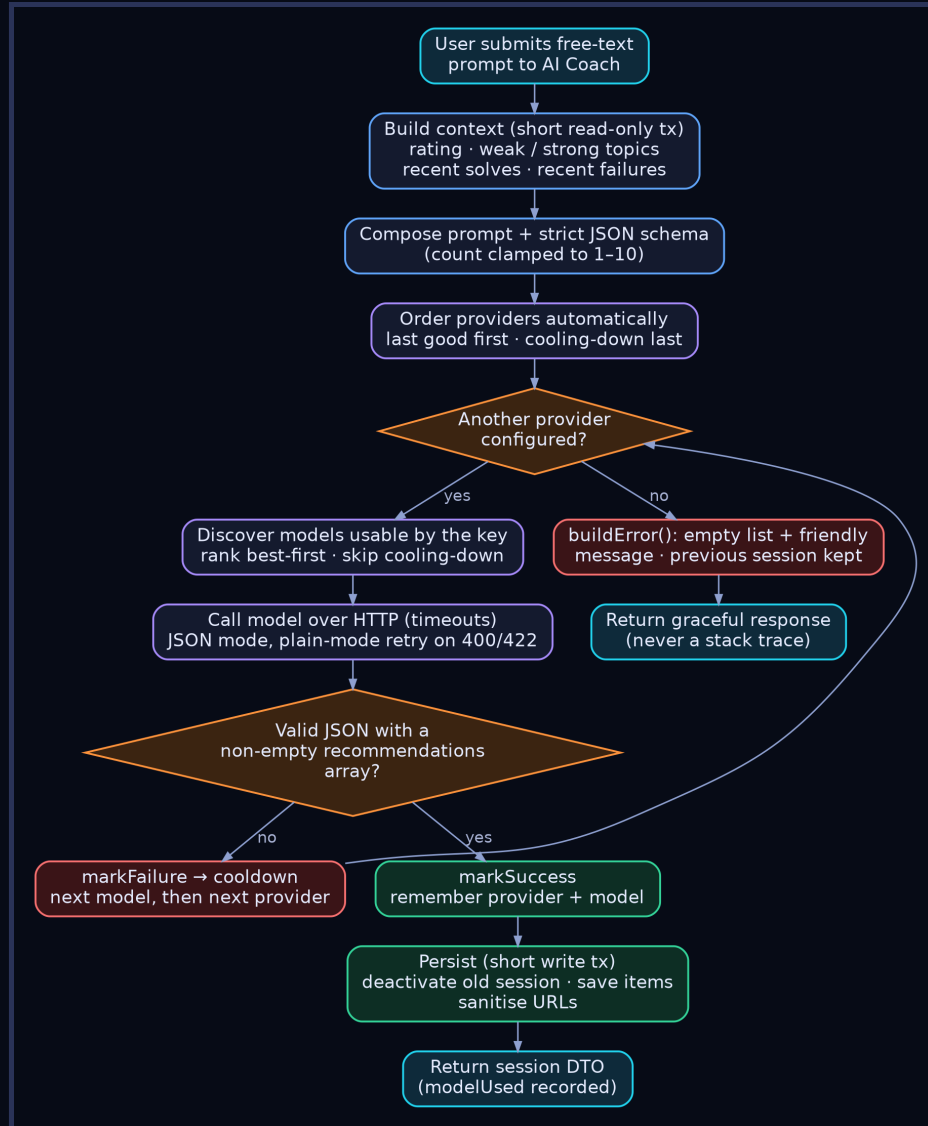
Assumptions and constraints

- Users supply public handles; no passwords needed
- Public APIs may rate-limit or change → one service per platform
- 200 fetched / sync, 300 retained → analytics show recent performance
- AI Coach needs internet and at least one provider key

Use Cases

UC	Use case	Key behaviour
UC-1	Account and handles	Register, login (JWT), save four platform handles
UC-2	Synchronise submissions	Manual or 6-hourly sync, deduplication, pruning, partial-failure isolation
UC-3	View analytics and insights	Rating, streaks, heatmap, weakness radar, verdicts, hidden insights
UC-4	Curated recommendations	Generate from weak topics, mark solved, dismiss
UC-5	AI Coach	Free-text goal → context + prompt → provider chain → saved session

Behaviour: AI Coach Activity Diagram



- Context built in a short read-only transaction
- Providers ordered automatically: last good first, cooling-down last
- Models discovered per key and tried best-first
- Only valid recommendation JSON is accepted
- Failure → cooldown → next model / provider
- Persist in a short write transaction; URLs sanitised
- All fail → friendly message, previous session kept

Data Model and Persistent Data

Entities (PostgreSQL on Neon)

- User — credentials, role, four handles
- Submission — verdict, language, time
- Problem — platform, rating, label; Topic (many-to-many)
- Recommendation — curated item + score
- AiRecommendationSession / Item — AI results

Integrity and access rules

- Deduplicate by platformSubmissionId
- Prune to 300 submissions per user (bulk DELETE)
- Open-session-in-view disabled
- JOIN FETCH queries + @Transactional(readOnly)
- SSL connection, HikariCP pool
- JPA repositories: 7 interfaces

Architecture

Presentation

- Next.js 14 · React 18
- Tailwind · Recharts
- Zustand (JWT) · TanStack Query / Axios

Application (Spring Boot 3.3.4)

- Controllers → Services → Repositories
- Spring Security · JWT filter
- @Async sync · @Scheduled 6 h
- AI provider layer

Data and external

- PostgreSQL (Neon, SSL)
- Codeforces · LeetCode · AtCoder · CodeChef
- Groq · Mistral AI · OpenRouter

Layered architecture; each platform and each AI provider is isolated behind its own service class.

Design Classes of the Problem Domain

Class	Responsibility / key attributes
User	id, username, email, passwordHash, role, codeforces/leetcode/atcoder/codechef handles, lastSyncedAt
Problem / Topic	platform, title, url, difficultyRating, difficultyLabel; topics (many-to-many)
Submission	user, problem, verdict (8 values), language, submittedAt, platformSubmissionId
Recommendation	user, problem, reason, score, solved, dismissed
AiRecommendationSession	user, prompt, coachInsight, focusAreas, modelUsed, active
AiRecommendationItem	session, title, platform, url, difficulty, topics, reason, solved, dismissed

Service-Layer Design Classes

Class	Role
AnalyticsServiceImpl	Combined analytics; sync and refresh (read-only transaction)
CombinedRatingCalculator	Per-platform ratings and weighted CodePulse Rating
Codeforces / Leetcode / AtCoder / CodeChef DataService	Independent platform sync services
SubmissionPruningService	Keeps the latest 300 submissions per user
RecommendationServiceImpl	Weakness-based curated recommendations
AiRecommendationServiceImpl	AI Coach orchestration over the provider chain
ScheduledSyncService	Cron job every 6 hours for all active users

Algorithm: CodePulse Rating

Platform	Per-platform rating	Cap
Codeforces	Actual rating, else estimate from solved difficulty	—
LeetCode	$1200 + 2 \cdot \text{easy} + 9 \cdot \text{medium} + 22 \cdot \text{hard} + \text{acceptance bonus}$	3200
AtCoder	Average difficulty of solved problems $\times 1.05$	—
CodeChef	$1000 + 3 \cdot \text{easy} + 10 \cdot \text{medium} + 20 \cdot \text{hard} + 35 \cdot \text{expert}$	3000

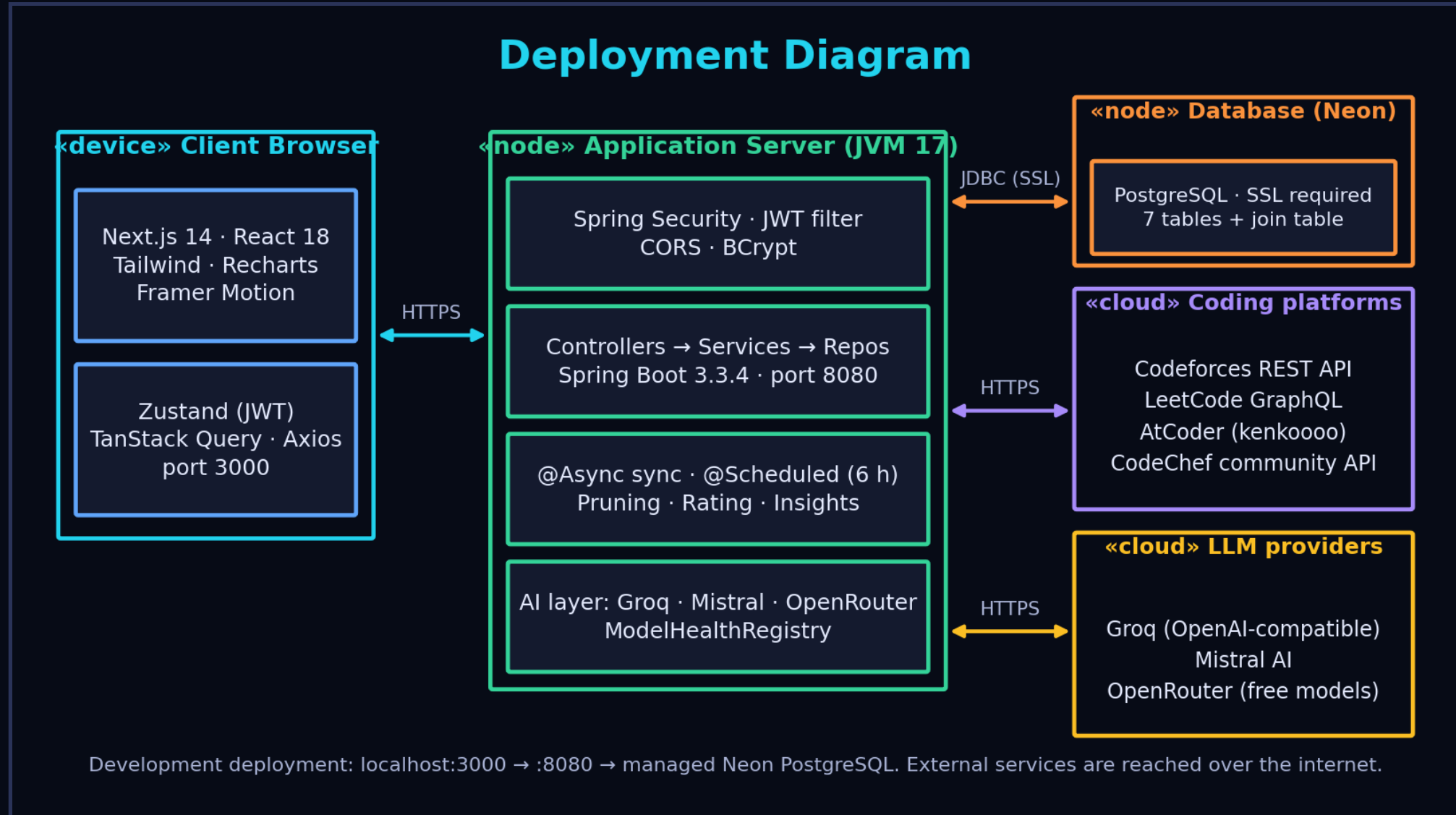
$\text{weight}(p) = \sqrt{\text{submissions}(p)}$ CodePulse Rating = $\frac{\sum \text{rating}(p) \cdot \text{weight}(p)}{\sum \text{weight}(p)}$, clamped to 800–3800

- Square-root weight: diminishing returns, no platform dominates
- Platforms without activity add zero weight — one platform still gives a meaningful rating

Algorithms: Weakness, Streak, Recommendations, Pruning

- $\text{Weakness}(\text{topic}) = \text{failures} / \text{attempts}$ — 0.0 strong ... 1.0 weakest
- Streak: submissions of all platforms merged into distinct days; current, longest, active days
- Recommendations: weak topics (failure rate $> 30\%$, top 3), 3 unsolved problems each, difficulty from rating – 200 (min 800) to rating + 300; fill to 10 by level
- Pruning: after each sync keep the newest 300 submissions (single bulk DELETE)
- Hidden insights: rule set in the frontend — critical, warning, success, tip, pattern findings
- Scheduled sync every 6 hours; one failing user never stops the batch

Deployment Diagram



Design Alternatives and Refactoring

Decision	Alternatives	Chosen because
LLM integration	Spring AI SDK · direct REST	SDK failed on additive response fields; REST is tolerant
Provider strategy	Single · primary + fallback · ordered chain	Free tiers rate-limit; a chain keeps the coach available
Model configuration	Fixed names · run-time discovery	Models are retired (404); discovery needs only the key
Reasoning models	Read reasoning field · require content	Reasoning text is not the answer
Transaction scope	One @Transactional · read tx + LLM outside + write tx	No DB connection held during a slow LLM call
Lazy loading	Open-session-in-view · JOIN FETCH + @Transactional	Predictable queries, no LazyInitializationException

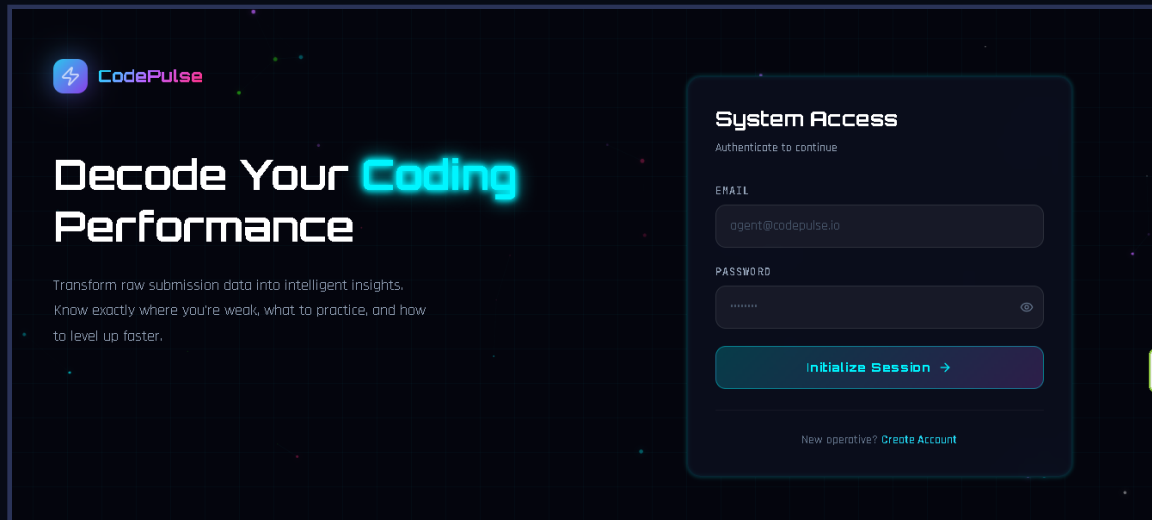
Interface Design: Pages and Objects

Route	Page	Main objects and actions
/auth	Login, 2-step registration	Email, password, step indicator, Initialize Session / Launch
/dashboard	Dashboard	Stat cards, rating card, platform connections, Sync All
/analytics	Deep Analytics	Insight cards, rating trend, radar, verdict donut, heatmap
/recommendations	Curated	Problem cards, platform filter, Solved / Dismiss
/ai-recommend	AI Coach	Prompt box, count 4/6/8, Generate, focus areas, cards
/profile	Profile	Basic info, four handle fields, Save

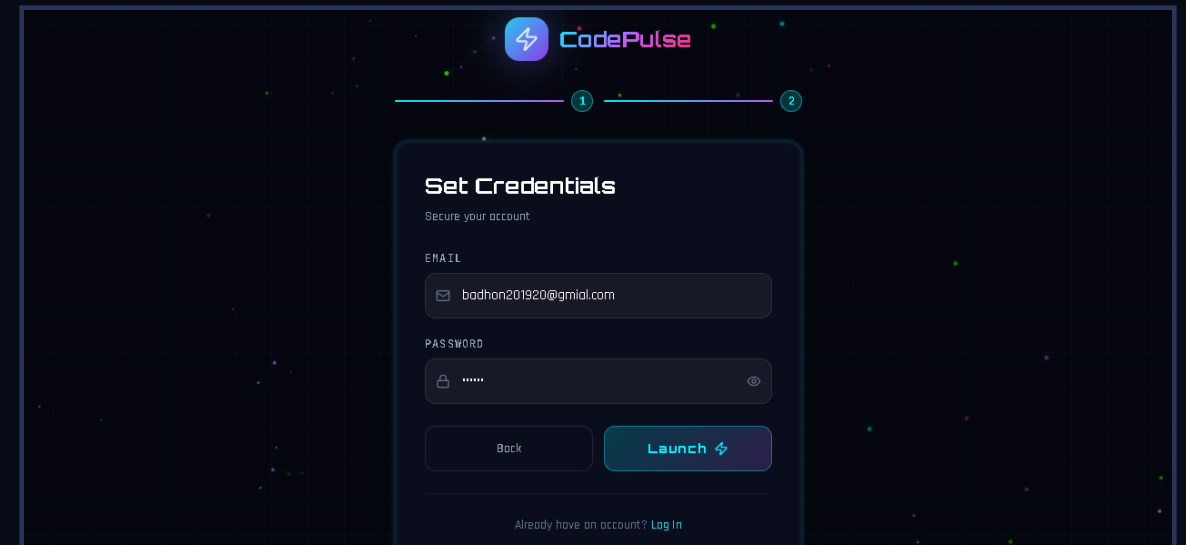
Interface Design: Events and State Changes

User event	State change
Submit login / registration	Button loading → dashboard, or error toast
Click Sync All Platforms	Spinner → data refreshed, last-synced time updated
Open Analytics	Skeleton → charts and insights
Generate recommendations	Skeleton → problem cards (or empty state)
Mark solved / Dismiss	Card removed from the list
Submit AI prompt	“Analysing your performance...” → results, or friendly error with previous session
Save handles	Toast confirmation; sidebar badges update

Running Application: Login and Registration



Login

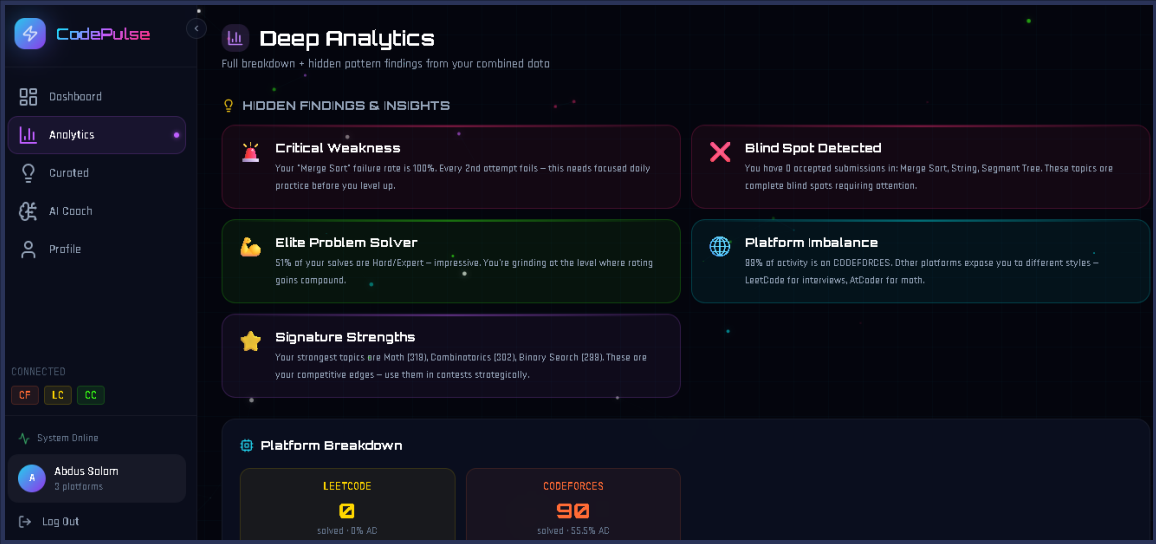


Registration — step 2

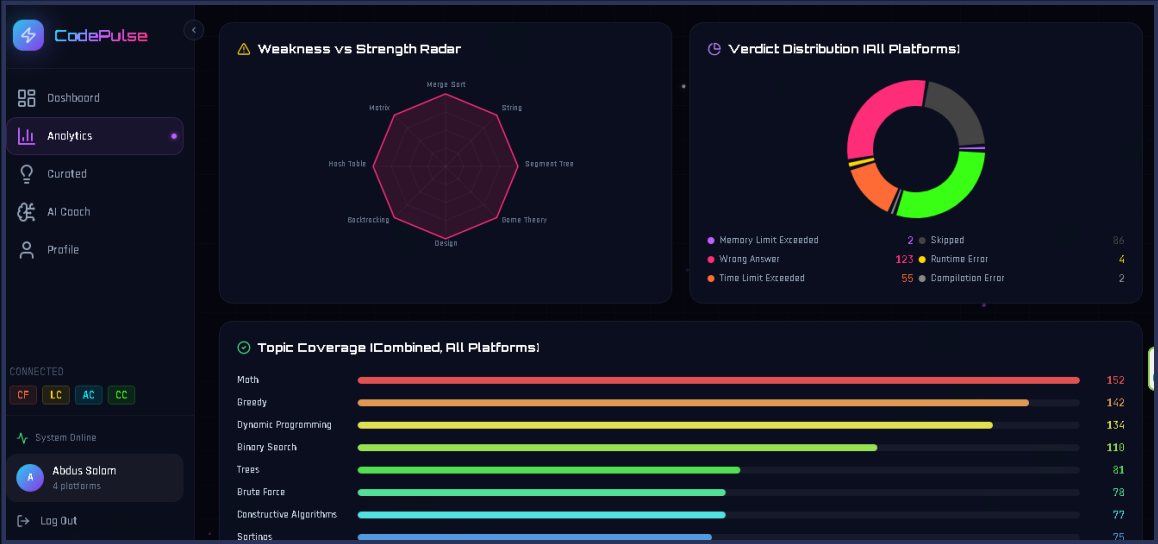
Running Application: Dashboard



Running Application: Analytics

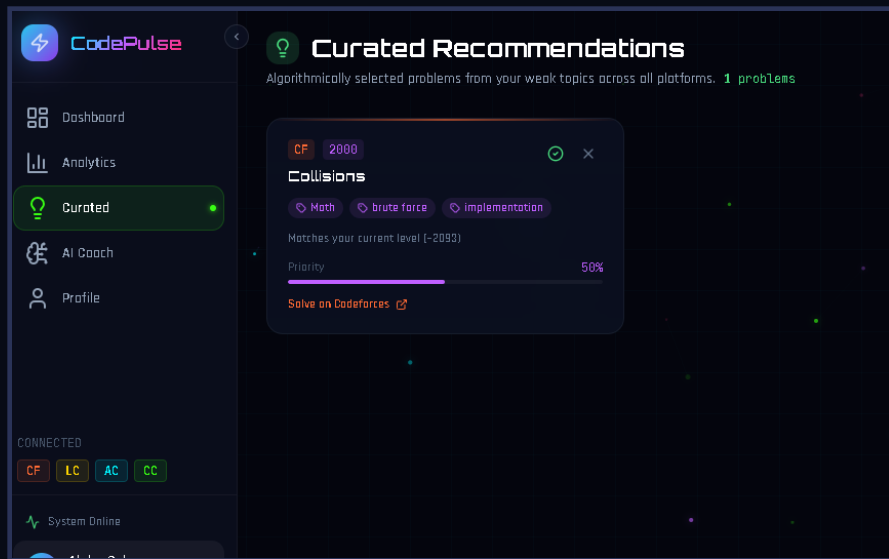


Hidden insights and platform breakdown

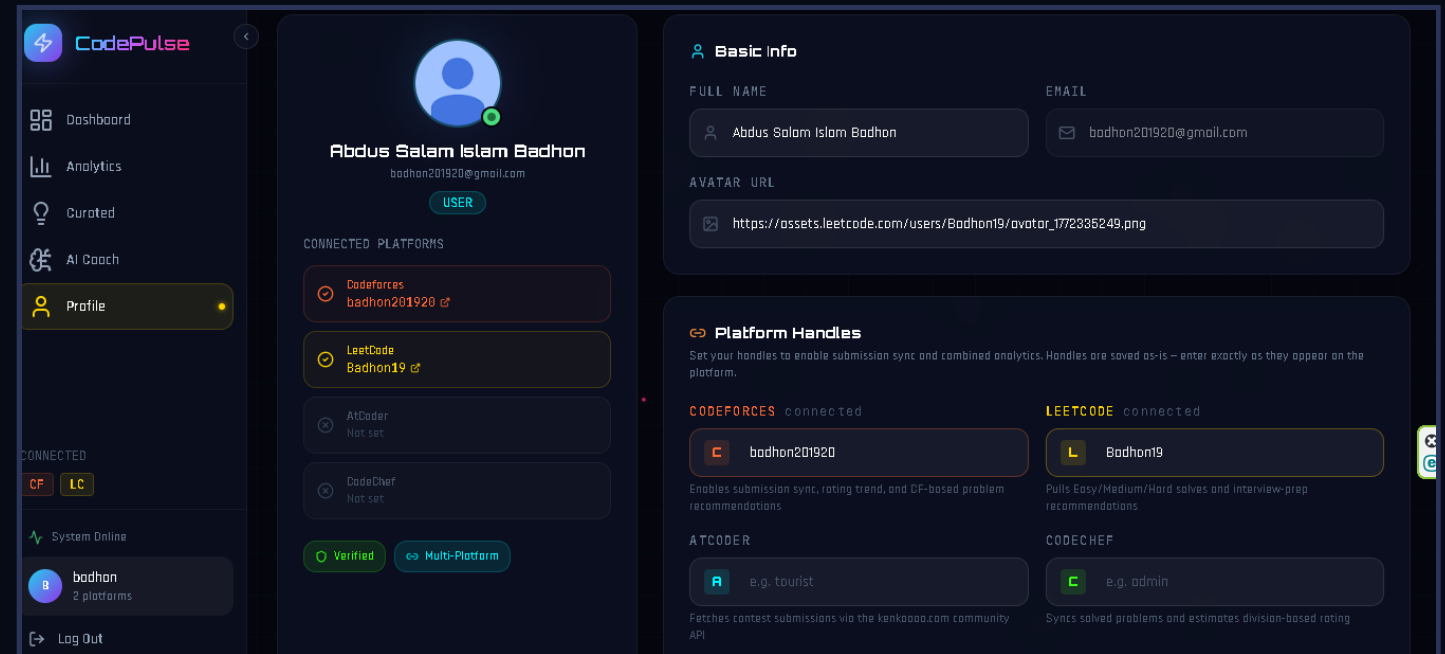


Radar, verdict distribution, topic coverage

Running Application: Curated Recommendations and Profile

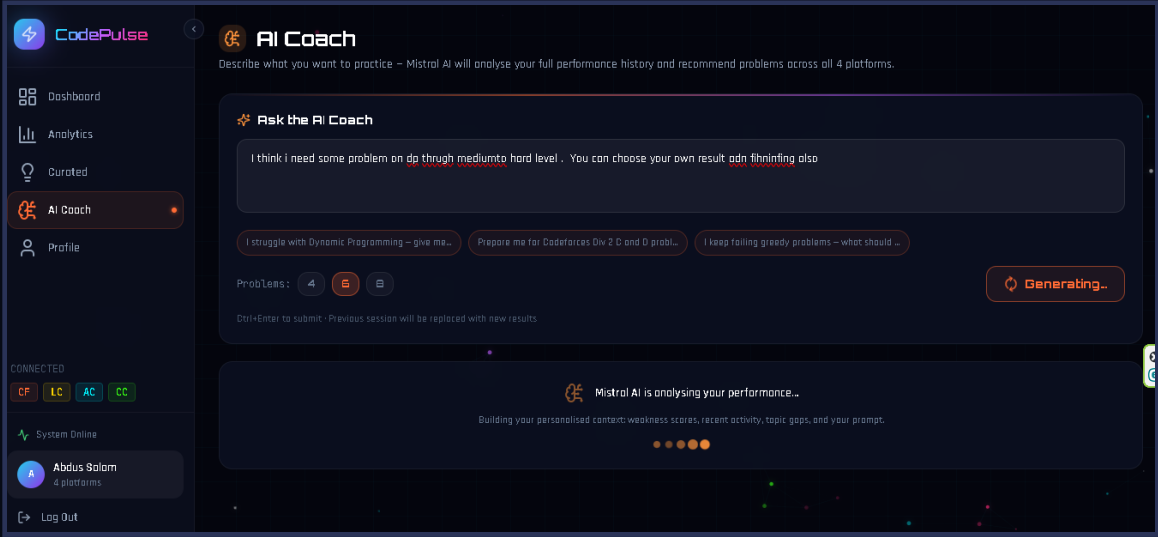


Curated recommendations

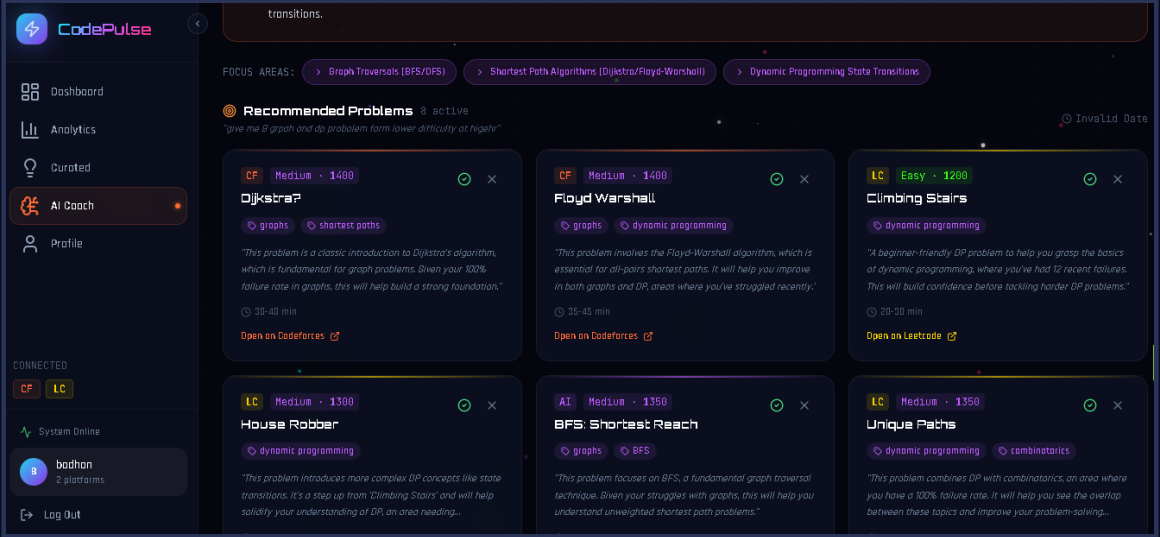


Profile and platform handles

Running Application: AI Coach



Prompt and “analysing” state

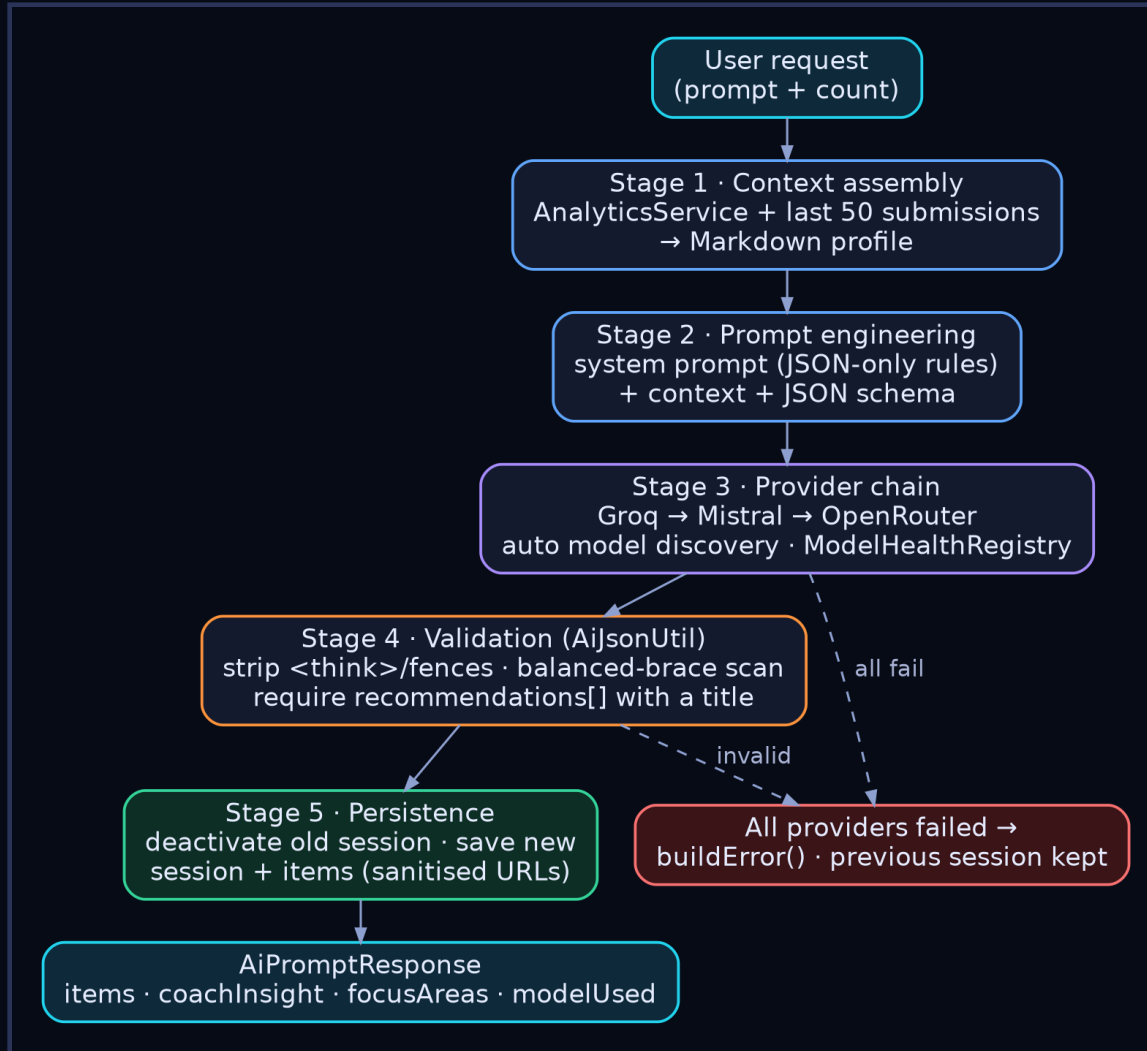


Recommended problems with focus areas and reasons

Implementation: Technology Stack

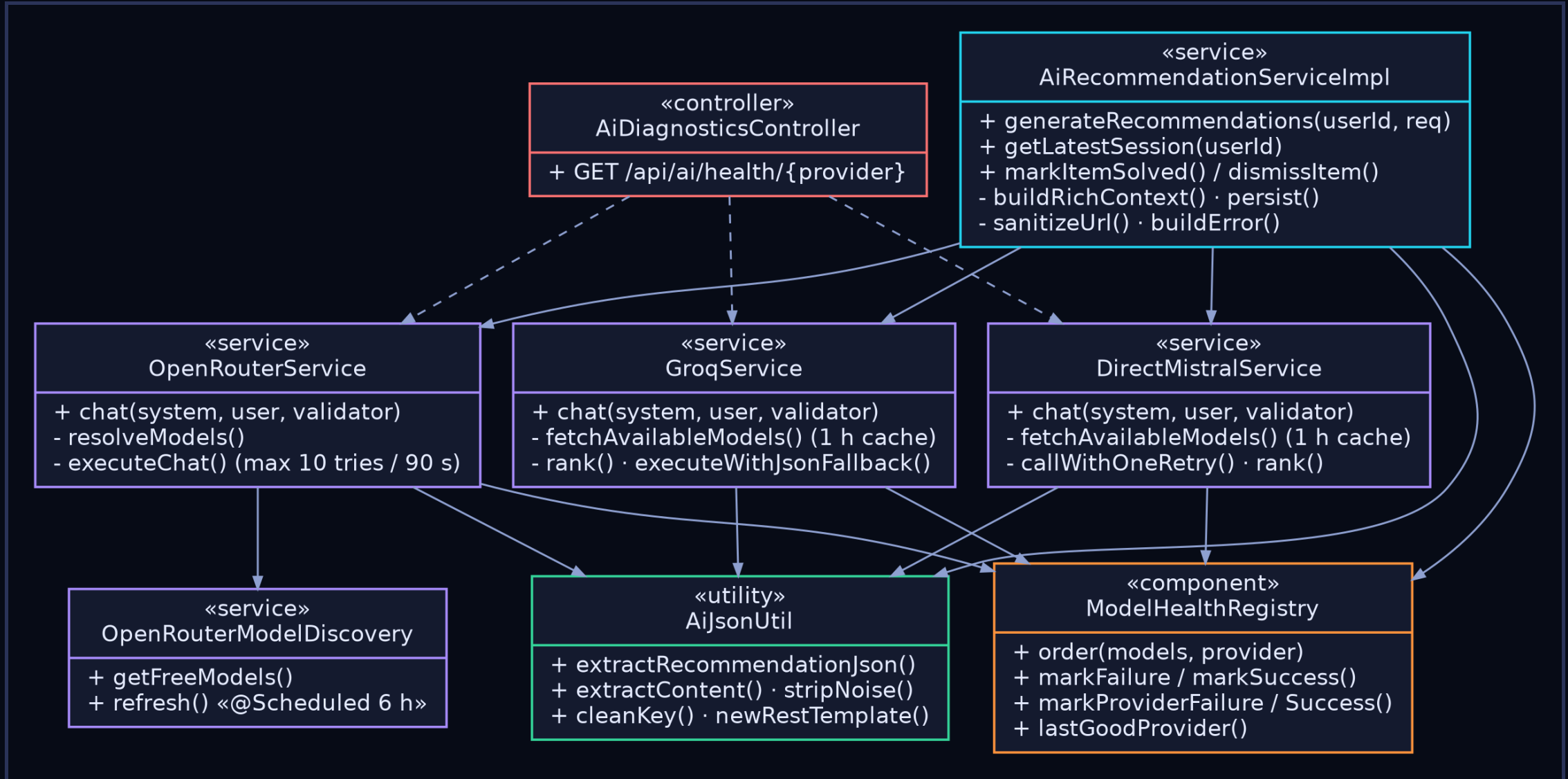
Layer	Technology
Backend	Java 17, Spring Boot 3.3.4, Spring Security (JWT HS256, BCrypt), Spring Data JPA, Hibernate 6.5.3, Maven
Database	PostgreSQL on Neon (SSL), HikariCP
Frontend	Next.js 14, React 18, Tailwind, Recharts, Framer Motion, Zustand, TanStack Query, Axios
Platform data	Codeforces REST, LeetCode GraphQL, AtCoder (kenkoooo), CodeChef community API
AI providers	Groq, Mistral AI, OpenRouter over direct HTTP (RestTemplate)
Concurrency	@Async sync services, @Scheduled 6-hourly job

AI Engineering: Pipeline



- Stage 1: context from analytics + last 50 submissions
- Stage 2: JSON-only system prompt, context, schema; count clamped to 1–10
- Stage 3: provider chain with model discovery
- Stage 4: AiJsonUtil — strips /fences, finds balanced JSON, requires titled recommendations
- Stage 5: sanitise URLs, save session and items
- Result includes the model used

AI Provider Layer: Design Classes



AI Resilience: Providers and Cooldowns

Provider	Model discovery and ranking	Limits
Groq	GET /models; gpt-oss-120b → 70b → gpt-oss-20b → llama-4 → qwen → kimi → 8b	10 s / 45 s; JSON mode, plain-mode retry
Mistral AI	/v1/models; small → nemo → minstral → medium → large	10 s / 60 s; one retry on 429 / 5xx
OpenRouter	Only :free models, text output, context ≥ 8k; non-reasoning first	≤ 10 attempts, 90 s total

Event	Groq	Mistral	OpenRouter
Unusable output	10 min	10 min	30 min
HTTP 429	3 min	3 min	3 min
HTTP 5xx	2 min	2 min	60 min
Other 4xx	60 min	60 min	60 min
HTTP 401 (bad key)	stop provider	stop provider	stop provider

Testing: Strategy, Criteria and Risks

Approach

- 17 automated end-to-end tests (JUnit 5, Spring Boot Test, TestRestTemplate)
- Real application on a random port, real database, real AI provider
- Regression guard: analytics, submissions, recommendations after JOIN FETCH fix

Pass / fail criteria

- PASS: expected status and JSON, no stack trace
- FAIL: any HTTP 500, LazyInitializationException, unprotected endpoint, unhandled AI error

Risks and contingencies

- Platform rate limits → retry later, per-platform isolation
- All free AI models unavailable → provider chain; saved session via `/api/ai/sessions/latest`
- Test users accumulate → separate test database, delete `e2e_` users

Test Cases and Results

Executed 10 October 2026 — 17 run, 0 failures, 0 errors, 0 skipped — BUILD SUCCESS

TC	Test case	Result
01	Register with valid data	PASS
02	Duplicate email rejected	PASS
03	Login with correct password	PASS
04	Login with wrong password	PASS
05	No token → denied	PASS
06	Get own profile	PASS
07	Update profile handles	PASS
08	Analytics for a new user	PASS
09	Recent submissions	PASS

TC	Test case	Result
10	Recommendations list	PASS
11	Generate without data	PASS
12	Admin endpoint as USER → 403	PASS
13	AI latest session	PASS
14	AI health endpoint	PASS
15	AI Coach request	PASS
16	AI count capped at 10	PASS
17	Sync starts asynchronously	PASS

Evidence: provider order [Groq, Mistral, OpenRouter]; Groq answered with openai/gpt-oss-120b in 3.2 s (TC-15) and 4.1 s (TC-16).

Conclusion, Deliverables and Q&A

- CodePulse unifies four platforms into one rating, analytics and insight dashboard
- Algorithmic and AI recommendations grounded in measured weaknesses
- Resilient AI layer: provider chain, model discovery, cooldowns, strict JSON validation
- Secure, bounded and tested: JWT + BCrypt, 300-row retention, 17 passing end-to-end tests
- Deliverables: report with user manual, repository github.com/Badhon1401/SPL-3, live demonstration, code review
- Installer: not applicable — run from source (installation guide in report)

Thank you — questions?